

Token & Words

1 token = 0.75 word ga tugri keladi. A 100 token esa tahminan 75 wordga tugri keladi

Tokenlar bilan wordlarni soni teng yoki unday bulmashi mumkin.

1-Case: token = words

2-Case: token > words

- uzun suzlarni kuplab tokenlarga buladi

3-Case: token < words

- bu holat kup kelmaydi

1, 2 Case – eng kup keladi

Juda kup LLM modellar english tilida training qilingan. Shunga misol uzbekcha bilan english tilining wordlar soni harhil chiqadi.

Biz bilamizki tokenization – tokenlarga ajratish usuli. lekin hohlaganday ajratmasdan modelni aqilliroq qilishni hisobga olib ajratish kerak.

Tokenization ning umumiy 4 ta usuli bor (Lesson 1 da aytilgan)

1. Whole-word tokenization:

- butunlay suzni tokenlash

2. Subword tokenization:

- manoga qaragan holda tokenlash. Misol 'un' suzlarni.

3. Character & Fragment tokenization:

- turli symbolarni tokenlash. Misol raqamlar yoki @ shunday symbolar.

4. Non-Word elements:

- bu suzlarnila emas. Symbolarniham suzlar qatorida olib ketadi. Yani birhil tokenlashadi

Tokenlardan foydalanish sabablari:

1. suzlarni control qila oladi. Chunki suzlar aslida limitlangan. Fine-tuning qilish imkoni bor

2. typoslarga yani suzda hatolik bulsaham uni tushunib uzi tugirlab uqish imkoni bor

3. GPU yani compute tomondan. Va uni costlarini tracking.

Model Training

Modelni training qilish uchun eng asosiy 6 da features:

1. **parameters**
 - $12 * L * D^2$
2. **memory** yetishi
 - $16 * B * N$
3. **compute budget** i
 - $6 * N * D$
4. **time** (wall-clock)
 - $6ND / (\text{GPU, peak, MFU})$
5. **cost**
 - time, GPU, \$ per hour
6. **samaradorlik**
 - MFU up, BF16, checkpoint

Example of Model Training

Architecture:

Layers (L): 24
d_model: 2048
Context Length: 2048
Precision: BF16 Compute / FP32 State
Optimizer: AdamW
Hardware: 4 × NVIDIA A100 (80GB)

Parameterlar soni:

$\approx 1.21\text{B}$ Parameters

$12 \times 24 \times 2048^2$ – shu formulaga kura 1.21B parameterlar soni chiqdi.

$N = 12 \times L \times d^2$ – formula

Weights (FP32): 4 B
Gradients (FP32): 4 B
Adam m (yunalish): 4 B
Adam v (kattalik): 4 B

Weights, Gradients, Adam m, Adam v – bular barchasi 4 B buladi. shunga $16B * 1.21B = 19.3 GB$ chiqadi. Natijada hotiraham yetadi degani.

Compute Budget: $C = 6 \times N \times D$

$6 * 1.21B * 24.2B$

-> 1.75×10^{20} FLOPs

Vaqt:
$$\frac{t = 6ND}{GPU * Peak * MFU}$$

$1.75e20 / (4 * 312e12 * 0.40)$

-> ~4 kun / ~97 hour

Cost: hour * GPU * \$ per hour

$97 \text{ hour} * 4 \text{ GPU} * \$1.2-2 / \text{hour}$

-> \$600 ~ \$800

Floating Point

Har bir son uzi float buladi. suzni raqamga convert qilamiz, usha convert bulingan raqamlar hammasi float. Ularning turli ulcham oraliq bor:

Format	Bits	Exp	Mantissa	Dynamic Range	Use Case
FP32	32	8	23	~1e-38 .. 1e38	Gold standard
FP16	16	5	10	~6e-5 .. 65504	Havfli
BF16	16	8	7	Same with FP32	Training matmul ideal
FP8	8	4/5	3/2	So limited	H100 only

Precision: ikki ketma-ket son orasida nechta qiymat mavjud

Tensors

Contiguous tensor: elementlari memoryda bitta blockda row-major tartibida save buladigon tensorlar. Kup PyTorch amallari contiguous tensor talab qiladi.

Non-contiguous tensor: $x.T$ dan transpose olingan bolsa usha non-contiguous tensorlar buladi

Einsum: matrix multiply, dot product, outer product va tensor. Errorlarni catch qiladi.

MFU: sekundiga bajarilgan haqiqiy foydali FLOPs. Qancha training bulsaham lekin sekundiga foydali ish qilmasa – yaxshi ishlamaydi.

MFU range	Evaluate
< 10%	Juda yomon, bottleneck
10 ~ 30%	Sub-optimal
30 ~ 50%	Normal
50 ~ 70%	Yaxshi
70 ~ 85%	Super
> 85%	Deyarli optimal holat (rare)

Optimizerlardan esa Adam bilan AdamW ni ishlatamiz. Qolganlari LLM uchun uncha mos emas.